Cliniface 7.0 Linux Build Process (Draft)

Disclaimer and License

Cliniface is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. Cliniface is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see http://www.gnu.org/licenses/.

The Cliniface repository can be found at:

https://github.com/frontiersi/Cliniface

Package building

If you are building Cliniface with the intention of creating a package for distribution, you may wish to do some from an older version of Ubuntu to maximize compatibility, with Ubuntu 16.04 the recommended version.

Linux distributions

The following examples are provided from within Linux Mint. Different Linux versions sometimes use different shells or slightly different directory structures. If you are not using Linux Mint, you will need to adapt any commands that don't work based on the distribution you are using.

Directory setup

Your home directory will depend on your operating system. If you are on linux, you can easily get to your home directory by opening a terminal and typing "**cd ~**". If you want to see the full location of your home directoy on Linux, you can then type "**pwd**". On Windows, your home directory should be stored under "**c:\Users**YourUsername"

Within your home directory, please create the following directory structure, where Home is a stand in for your home directory location:

Home/local_source

Home/local_libs

Home/local_build

Home/dev

Home/dev/libs

Home/pylibs

Home/bin

To make CLI navigation a little easier, it's worth adding some symbolic links to make changing directories quicker. From your home directory, you can do this with the following commands:

```
sudo ln –s ls local_source
sudo ln –s ll local_libs
sudo ln –s lb local_builds
```

Now you can use 'cd ~/ls' to get to your local_source directory, etc.

Required libraries

You will need to install several libraries that are either used by Cliniface, or one of its dependencies. The following command will attempt to install them all, but you use the command at your own risk:

```
sudo add-apt-repository "deb http://security.ubuntu.com/ubuntu xenial-security main"sudo apt update
sudo apt-get install git-lfs libz-dev libjpeg-dev libwebp-dev libpng-dev libtiff-dev libjasper1 libjasper-dev libilmbase-dev libopenexr-dev libgstreamer1.0-dev libgtk2.0-dev libgtk-3-dev libdc1394-dev libzzip-dev texlive-full build-essential
```

The command above attempts to install the following list of libraries:

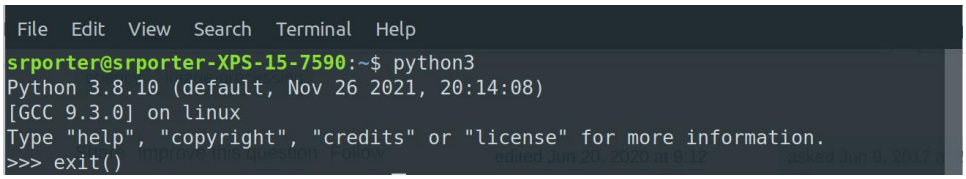
git-lfs	libz-dev	libjpeg-dev
libwebp-dev	libpng-dev	libtiff-dev
libjasper1	libjasper-dev	libilmbase-dev
libopenexr-dev	libgstreamer1.0-dev	libgtk2.0-dev
libgtk-3-dev	libdc1394-dev	libzzip-dev
texlive_full	build-essential	

To install libjasper you will need to add the following repository:

```
sudo add-apt-repository "deb http://security.ubuntu.com/ubuntu xenial-security main"
```

Python

You will require Python for the Cliniface Install. You can check if you already have python installed by typing **python3** from a terminal, if it's installed you will be presented with a window similar to the following:



You can see from this prompt that this computer is running Python 3.8.10. As long as you have Python 3.5 or later installed, Cliniface should work fine.

If python is not installed, you can install it with:

```
sudo apt-get install python3.8
```

With this command installing Python 3.8.

Qt5

You will need to acquire Qt5, which can be installed by going to https://www.qt.io/. On the website at the time of writing this guide, you want to click "**Browse Qt Tools**", scroll to the bottom of the page and click "**Get your free Qt trial**", then scroll to near the bottom of the page and click "Go open source". Scroll to near the bottom of the Open source page and click "**Download the Qt Online Installer**", then "**Download**".

This will download the Qt Online installer, which when complete we will need to run to install several tools we need. Once downloaded, you will need to chmod the qt installer to be an executable with the command **chmod +x <qtinstaller>**, then run the executable and you will be asked to sign into your Qt account. If you don't already have an account, follow the link provided in the installer to sign up, don't worry, it's free!

Once you've signed in, you will need to read and accept some agreements and click next a few times until you get to the Installation Folder page. Specify **Home/Qt** as the install directory, and select **Custom Installation**. You can install it elsewhere if you want, but instructions later in this guide will assume this location so if you install it elsewhere you will need to apply those changes yourself to later instructions.

You will need the following components and ONLY the following components for the Cliniface installation:

Qt 5.15.2 > MSVC, MinGW, Qt Charts, and Qt Data Visulisation only.

Developer and Designer Tools > Qt Installer Framework, CMake, Ninja, and OpenSSL.



Finish the install process and Qt will download which may take awhile. While it's installing, you can move on and install other required software, or grab a coffee.

Required libraries

In order to build Cliniface, you will need the following libraries/binaries:

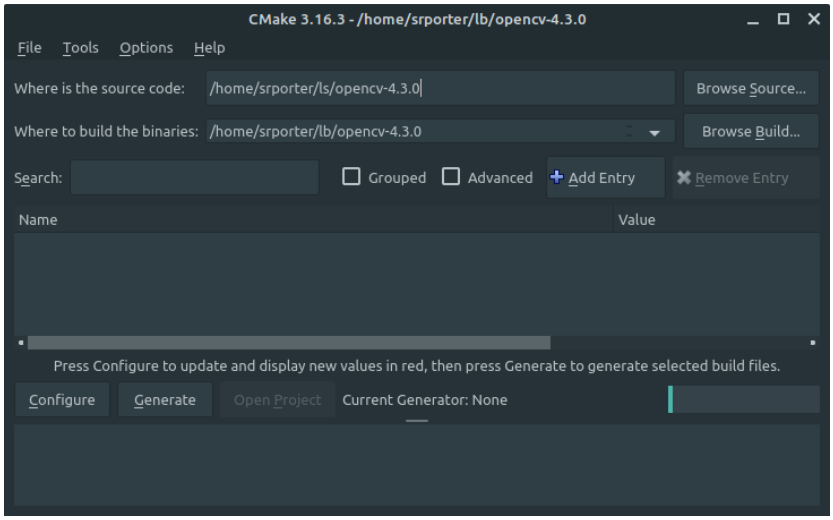
VTK 9.1.0
AssImp 5.1.6
Boost 1.73
OpenCV 4.3 with OpenCV_contrib 4.3.0
Eigen 3

Lua 5 with Sol2 header
Nanoflann
Quazip

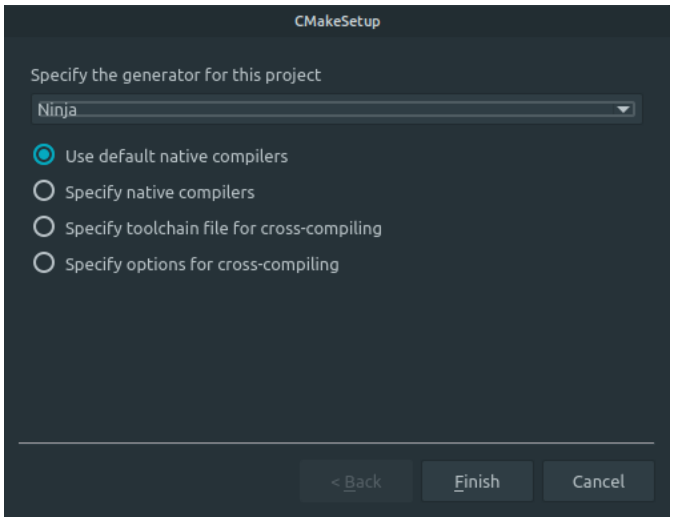
Which should all be placed in "**Home/local_libs**" so the various packages can find the components they need. Note: AssImp needs to be placed in **HOME/local_libs/AssImp**, and VTK needs to be installed to **HOME/local_libs_VTK** specifically.

These packages can be difficult to install, and it's not in the scope of this guide to provide instructions for installing each of them. We will provide some instructions below, but search engines, forums, and your own ingenuity will have to help you through the rest.

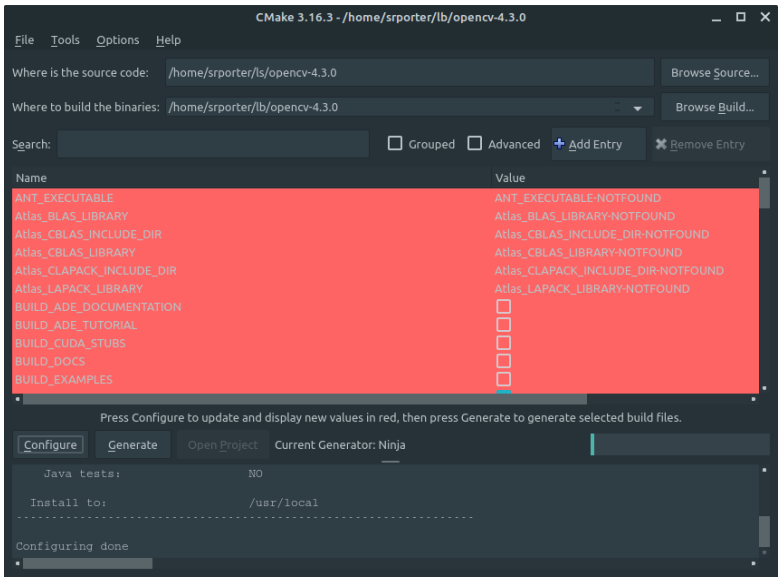
As an example, we'll look at the process for building OpenCV 4.3.0. You should be able to acquire the source code from www.github.com/opencv/, which you should then unzip into **Home/local_source**. You will also want to acquire OpenCV_contrib 4.3.0, and unzip this into your **Home/local_source** as well. If you then open your **Home/local_source** directory in a terminal, and use 'cd opencv-4.3.0' to change to the directory then run 'cmake-gui .' which will present you with a window similar to the following:



You will need to tell the CMake gui where to find the source, as well as where to build the file to (Home/local_build/opencv-4.3.0). If you then click the 'Configure' button, the interface will present you with a prompt asking what tools you want to use. You should specify 'Ninja' for the compiler:



After you hit Finish the top pane of the Cmake-gui should fill with text that is highlighted in red. The red highlight indicates that this is a new setting, which will help you identify when a change to one setting makes a new setting available. We need to alter several of these settings, with the easiest way to find them be using the search bar.



Set the following values:

BUILD_opencv_world	ON
WITH_QT	OFF
BUILD_opencv_python_bindings_generator	ON
BUILD_opencv_python_tests	ON
BUILD_CUDA_STUBS *through to* BUILD_ZLIB	OFF, except following

BUILD_IPP_IW	ON
BUILD_ITT	ON
BUILD_JAVA	ON
BUILD_PACKAGE	ON
BUILD_PROTOBUF	ON
BUILD_SHARED_LIBS	ON
BUILD_PERF_TESTS	OFF
BUILD_TESTS	OFF
OPENCV_EXTRA_MODULES_PATH	HOME/local_source/opencv_contrib-4.x/modules
CMAKE_INSTALL_PREFIX	HOME/local_lib/opencv-4.3.0/

After the values are set, hit **Configure** and the list will refresh. Check you're happy with all the values, then press **Generate**. This will create the build file in HOME/local_build/opencv-4.3.0, open a terminal and move to this directory, then use the command:

```
Cmake --build .
```

Cmake will then use the build file to build all the necessary files. When it's done, check the output to make sure it completed successfully, otherwise you will need to try and work through any issues raised. Once it does complete successfully, you will then need to run the install command:

```
Cmake --install .
```

Cmake will again output as it goes, and the process will take awhile for opencv. Again when it's done, check the output to ensure it completed successfully. If it didn't, you will need to troubleshoot. If it did, great! Move onto the next one.

Github Clone

Next you will need to clone the various repositories from Github into the required directories. You can do this via several means, but we will describe how to do this from the terminal using Git.

You will need to acquire:

libbuild	https://github.com/richeytastic/libbuild
rlib	https://github.com/richeytastic/rlib
rimg	https://github.com/richeytastic/rimg
r3d	https://github.com/richeytastic/r3d
r3dio	https://github.com/richeytastic/r3dio
r3dvis	https://github.com/richeytastic/r3dvis
rNonRigid	https://github.com/richeytastic/rNonRigid
QTools	https://github.com/richeytastic/QTools
FaceTools	https://github.com/frontiersi/facetools

Cliniface	https://github.com/frontiersi/Cliniface

To do this from your terminal, navigate to **Home/dev/libs** and use the following command:

```
git clone https://github.com/richeytastic/libbuild.git
```

This will begin the process of cloning the libbuild components into Home/dev/libs/libbuild. Once it is done, repeat the process for rlib through to FaceTools:

```
git clone https://github.com/richeytastic/rlib.git

git clone https://github.com/richeytastic/rimg.git

…

git clone https://github.com/richeytastic/qtools.git

git clone https://github.com/richeytastic/facetools.git
```

Note: The "**…**" above is a stand in for the other git clone commands.

Once all the supporting components have been cloned from Github, move up a directory to **Home/dev** and then clone Cliniface

```
git clone https://github.com/frontiersi/Cliniface.git
```

Environment Setup

Cliniface will need a few environment variables set in the Terminal in order to build and run correctly. Use your preferred editor to open **Home/.profile**, and **add the following bolded lines** to the start of the file:

```
…
#umask 022

export DEV_PARENT_DIR=~/dev/libs
export BUILD_PARENT_DIR=~/local_builds
export INSTALL_PARENT_DIR=~/local_libs

export BOOST=~/ll/boost/boost_1_73_0
export QT5=~/Qt/5.15.2/gcc_64
export VTK=~/ll/VTK

# if running bash
…
```

And **add the following bolded lines** to the end of the file:

```
…
export PATH="${PATH}:${HOME}/.local/bin:${QT5}/bin:${HOME}/bin"
```

```
export LD_LIBRARY_PATH="${LD_LIBRARY_PATH}:${BOOST}/lib:${VTK}/lib"
export PYTHONPATH="${PYTHONPATH}:${HOME}/pymodules"
```

Build components

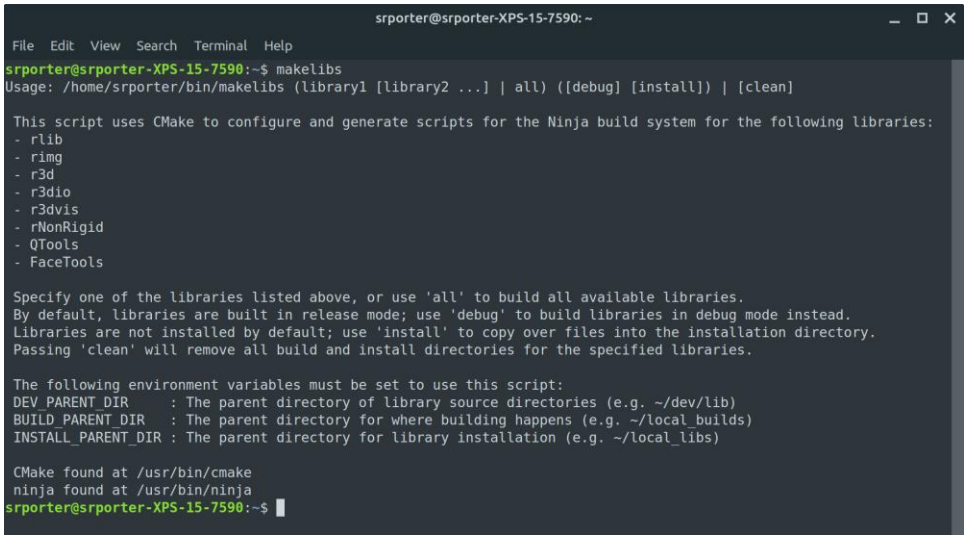
To run the components correctly you will need to set up a symbolic link for the makelibs.py file located in **Home/dev/libs/libbuilds/**, which you can do with the following command:

```
ln -s  ~/dev/libs/libbuild/makelibs.py ~/bin/makelibs
```

If you type the command:

```
makelibs
```

You should get the following output:



If not, there's an issue. Go back and check that you setup the Environment variables correctly, and named all the directories consistently.

We are now ready to try and build all the components! You can now type:

```
makelibs all install
```

And the libbuild component will run it's scripts and attempt to build all the components. This will take awhile and produce a lot of output. You may prefer to specify a single library, such as rlib, and build one at a time. If so, you should do so in the order they are listed in the terminal documentation, as later libraries such as r3dio make use of earlier libraries such as r3d.

Either way, check the output generated by the build script for any errors. Some warnings will be created, such as "loss of accuracy", which can be ignored, we are only concerned with any FAILED messages that indicate an error that prevent compilation. If such an error has occurred, you can use:

makelibs all clean

This command will remove all the currently compiled builds so you can recompile them all or one at a time to identify the issue preventing completion. Boost is commonly a source of problems, as it's a complex library. The error messages produced should indicate to you what library failed and why, and you will need to google and work to resolve the issue, then re-build that library and any that rely upon it.

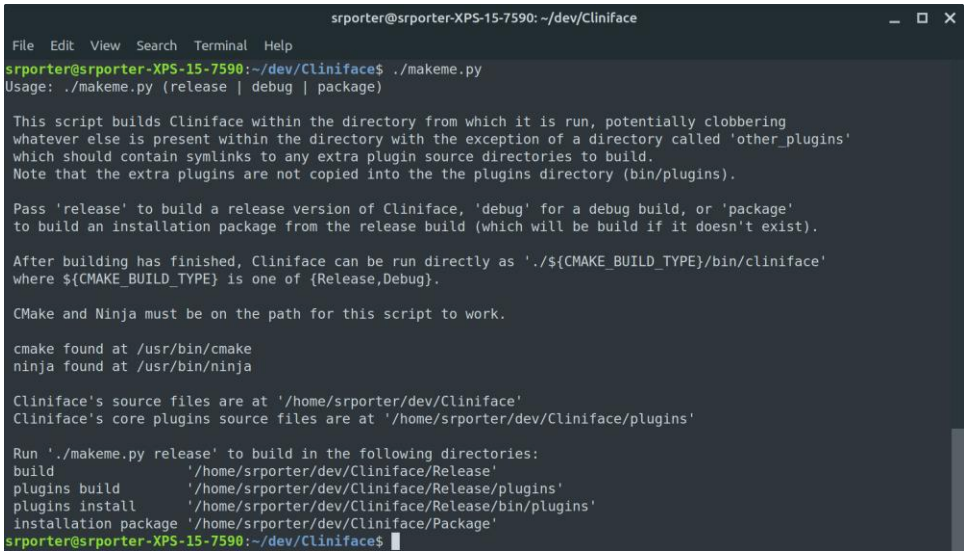
Build Cliniface

Once all the libraries have been built on your system, you can attempt to build Cliniface itself. If everything else has compiled without error, this process will most likely go smoothly.

Before you build Cliniface, you need to set up its directory. From **Home/local_builds** in a terminal, run the command:

mkdir Cliniface

Then change directory to **Home/dev/Cliniface** and run the command "**makeme.py**":

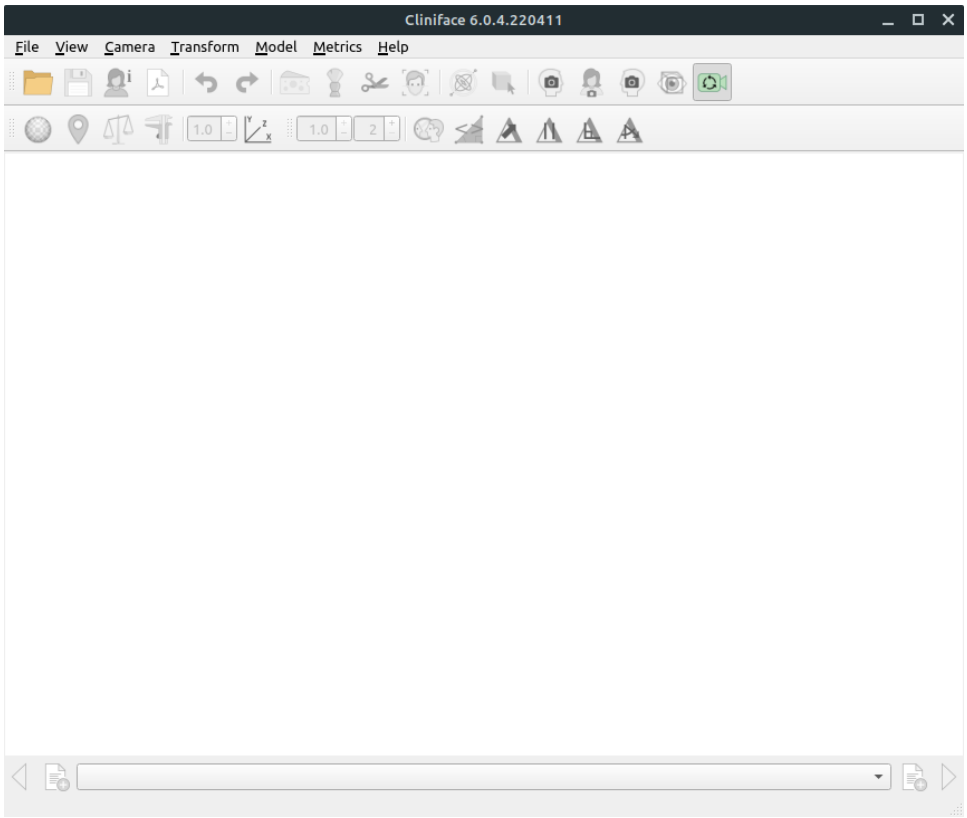


Then run "**makeme release**", and the Cliniface build scripts will go through and build the Cliniface binaries. The build process shouldn't take too long, once done check for any error messages. If everything finished installing without error, you are now ready to run Cliniface!

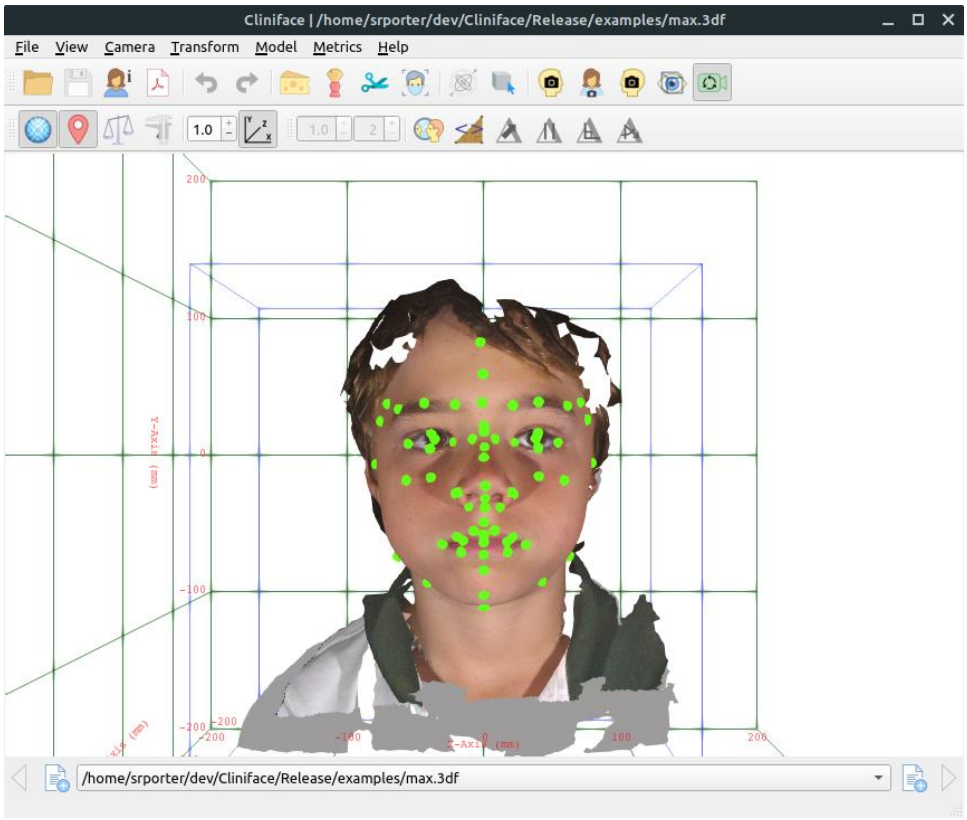
From **Home/dev/Cliniface/Release/bin/**, run the command:

./cliniface

And Cliniface should load and present you with it's GUI:



You can load a sample dataset to test the functionality from **Home/dev/Cliniface/Release/examples**, such as the max.3df dataset:


Cliniface | /home/srporter/dev/Cliniface/Release/examples/max.3df
File View Camera Transform Model Metrics Help
1.0
1.0
2
200
100
0
-100
-200
-200
-100
0
100
200
Y-Axis (mm)
X-Axis (mm)
/home/srporter/dev/Cliniface/Release/examples/max.3df